

OOPs

1. OOP - Core Idea

Object-Oriented Programming models software as a collection of **objects**.

An object has:

- **State** → data/attributes
- **Behavior** → functions/methods
- **Identity** → a distinct object in memory

Example:

```
class BankAccount {  
    private:  
    double balance;  
    public:  
    void deposit(double amount) {  
        balance += amount;  
    }  
    double getBalance() {  
        return balance;  
    }  
};
```

Here:

- `balance` → state
- `deposit()` → behavior
- A particular `BankAccount` instance → object

```
BankAccount account1;  
BankAccount account2;
```

`account1` and `account2` are two different objects.

2. Class vs Object

Class

A **blueprint/template**.

```
class Car {  
public:  
    string brand;  
    void drive() {  
        cout<<"Driving";  
    }  
};
```

Object

An actual instance of the class.

```
Car car1;  
Car car2;
```

Think:

```
Class  
↓  
Blueprint  
  
Object  
↓  
Actual thing created from blueprint
```

3. Constructor

A constructor is a special member function that is automatically called when an object is created.

Properties:

- Same name as class
- No return type
- Automatically called
- Can be overloaded

```
class Student {  
private:  
string name;  
int age;  
public:  
Student(string n,int a) {  
name=n;  
age=a;  
}  
};
```

```
Students("Ali",20);
```

4. Default Constructor

Constructor with no parameters.

```
class Student {  
public:  
Student() {  
cout<<"Default constructor";  
}  
};
```

```
Students;
```

5. Parameterized Constructor

```
class Student {  
public:  
    string name;  
    Student(string n) {  
        name =n;  
    }  
};
```

```
Students("Ali");
```

6. Constructor Overloading

Multiple constructors with different parameters.

```
class Student {  
public:  
    Student() {  
        cout<<"Default";  
    }  
    Student(string name) {  
        cout<<name;  
    }  
    Student(string name,int age) {  
        cout<<name<<" "<<age;  
    }  
};
```

7. Member Initializer List

Preferred way to initialize members.

```
class Student {  
private:  
    string name;  
    int age;  
public:  
    Student(string n, int a): name(n), age(a) {  
        }  
};
```

Instead of:

```
Student(string n, int a) {  
    name=n;  
    age=a;  
}
```

Why initializer list?

It directly initializes members instead of default-constructing and then assigning.

It is **required** for:

- `const` members
- reference members
- members without default constructors

Example:

```
class A {  
    const int x;  
    int& ref;  
public:
```

```
A(int value, int &r): x(value), ref(r) {}  
};
```

8. Destructor

Destructor is called when an object is destroyed.

Syntax:

```
~ClassName()
```

Example:

```
class Student {  
public:  
    ~Student() {  
        cout<<"Destructor called";  
    }  
};
```

Destructor:

- Has same name as class
- Starts with `~`
- Has no return type
- Takes no parameters
- Cannot be overloaded

9. Constructor vs Destructor

Constructor	Destructor
Initializes object	Cleans up object
<code>ClassName()</code>	<code>~ClassName()</code>

Constructor	Destructor
Can be overloaded	Cannot be overloaded
Called when object created	Called when object destroyed
Can take parameters	Takes no parameters

10. **this** Pointer

this is a pointer to the **current object**.

```
class Student {  
private:  
    string name;  
public:  
    Student(string name) {  
        this->name=name;  
    }  
};
```

Here:

```
this->name
```

means current object's **name** .

Returning **this**

```
class Counter {  
private:  
    int value=0;  
public:  
    Counter& increment() {  
        value++;  
        return *this;  
    }  
};
```

```
    }  
};
```

Then:

```
Counter c;  
c.increment().increment().increment();
```

11. Static Data Member

A static data member belongs to the **class**, not each individual object.

```
class Student {  
public:  
    static int count;  
    Student() {  
        count++;  
    }  
};  
int Student::count=0;
```

```
Student a;  
Student b;  
cout<<Student::count;// 2
```

There is only **one shared** `count`.

12. Static Member Function

```
class Student {  
private:  
    static int count;  
public:
```



```
static int getCount() {  
    return count;  
}  
};  
int Student::count=0;
```

Call:

```
Student::getCount();
```

Important interview point

A static member function does **not have a `this` pointer**.

Therefore it cannot directly access non-static members.

13. The 4 Pillars of OOP

You absolutely need these for interviews:

1. **Encapsulation**
2. **Abstraction**
3. **Inheritance**
4. **Polymorphism**

4 Concepts of OOP



Encapsulation



Abstraction



Inheritance



Polymorphism



Encapsulation

Data hiding & security



Inheritance

Reuse & extend existing code



Polymorphism

One interface, many implementations



Abstraction

Focus on what, not how

14. Encapsulation

Encapsulation means:

Bundling data and the methods that operate on that data together, while controlling access to the internal state.

Example:

```
class BankAccount {  
    private:  
    double balance = 0;  
    public:  
    void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
    double getBalance() {  
        return balance;  
    }  
};
```

You cannot directly do:

```
account.balance = -100000;
```

because `balance` is private.

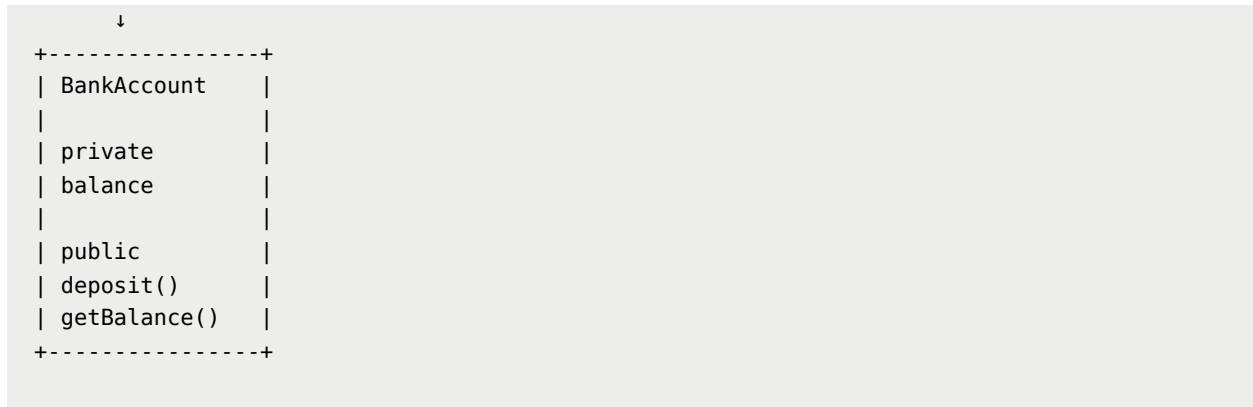
Instead:

```
account.deposit(5000);
```

Why?

Because the class controls how its state changes.

```
Outside world  
|  
| deposit()
```



Interview answer

Encapsulation is the bundling of data and behavior inside a class while restricting direct access to internal state using access modifiers such as `private`, `protected` and `public`.

15. Access Modifiers

C++ has:

private

Accessible only inside the class.

```
class A {  
    private:  
    int x;  
};
```

public

Accessible from anywhere through the object.

```
class A {  
    public:  
    int x;  
};
```

protected

Accessible:

- inside the class
- inside derived classes

```
class Parent {  
protected:  
int x;  
};  
class Child :public Parent {  
public:  
void test() {  
x=10;  
}  
};
```

16. Abstraction

Expose what an object does while hiding how it does it.

For example:

```
car.start();
```

You don't care about:

```
fuel injection  
spark  
engine timing  
battery  
starter motor
```

You only interact with:

```
start()
```

17. Abstraction in C++

A common way is through **abstract classes and pure virtual functions**.

```
class Payment {  
public:  
virtual void pay(double amount)=0;  
};
```

`= 0` means **pure virtual function**.

You cannot instantiate:

```
Payment p; // ERROR
```

Instead:

```
class CreditCardPayment : public Payment {  
public:  
void pay(double amount) override {  
    cout << "Paid using credit card\n";  
}  
};
```

Now:

```
Payment* payment = new CreditCardPayment();  
payment->pay(500);
```

The caller doesn't need to know how payment occurs.

18. Encapsulation vs Abstraction

Very common interview question.

Encapsulation

Focuses on:

| **How do I protect/bundle data?**

Example:

```
private:  
double balance;
```

Abstraction

Focuses on:

| **What should the outside world see?**

Example:

```
virtual void pay()=0;
```

Simple distinction:

```
Encapsulation → Hide data/internal state  
Abstraction   → Hide implementation complexity
```

19. Inheritance

Inheritance allows one class to acquire properties and behavior from another class.

```
class Animal {  
public:
```

```

void eat() {
    cout<<"Eating\n";
}
};
class Dog :publicAnimal {
public:
void bark() {
    cout<<"Barking\n";
}
};

```

Now:

```

Dog d;
d.eat();
d.bark();

```

`Dog` inherits from `Animal`.

```

Animal
|
↓
Dog

```

20. Types of Inheritance

Single

```

A
|
B

```

```

classB :publicA {};

```


Multilevel

```
A
|
B
|
C
```

Hierarchical

```
  A
 / \
B   C
```

Multiple

```
  A   B
 \   /
  C
```

```
class C : public A, public B {};
```

Hybrid

Combination of multiple inheritance types.

21. Composition vs Inheritance

This is **extremely important**.

Suppose:

```
class Engine {
public:
void start() {
}
```

```
};  
class Car {  
private:  
    Engine engine;  
};
```

This represents:

| Car **has an** Engine.

That's **composition**.

```
Car  
|  
+-- Engine
```

Inheritance represents:

| Dog **is an** Animal.

```
Animal  
  ↑  
Dog
```

Rule

```
IS-A → Inheritance  
HAS-A → Composition
```

Prefer composition when inheritance doesn't represent a true subtype relationship.

22. Public vs Protected vs Private Inheritance

```
class Child :public Parent
```

Public inheritance represents an **is-a relationship**.

Example:

```
Dog IS-A Animal
```

Access transformation:

Public inheritance

```
Parent public    -> Child public  
Parent protected -> Child protected  
Parent private   -> inaccessible
```

Protected inheritance

```
Parent public    -> Child protected  
Parent protected -> Child protected  
Parent private   -> inaccessible
```

Private inheritance

```
Parent public    -> Child private  
Parent protected -> Child private  
Parent private   -> inaccessible
```

23. Association

Association means:

Two objects know about or interact with each other.

Example:

```
class Teacher {
public:
void teach() {
}
};
class Student {
public:
void learn() {
}
};
```

A teacher teaches a student.

Teacher ----- Student

Neither necessarily owns the other.

24. Aggregation

Aggregation is a **weak HAS-A relationship**.

Example:

```
class Student {
};
class Department {
private:
vector<Student*> students;
public:
void addStudent(Student*s) {
students.push_back(s);
}
};
```

The department contains students, but students can exist independently.

```
Department
|
+----- Student
+----- Student
```

Students can exist without Department

25. Composition

Composition is a **strong HAS-A relationship**.

```
class Engine {
};
class Car {
private:
Engine engine;
};
```

The engine belongs to the car.

Conceptually:

```
Car
|
+--- Engine
```

When the `Car` is destroyed, its contained `Engine` is destroyed too.

Difference

```
Association
↓
knows/uses

Aggregation
↓
has, but weak ownership
```

Composition
↓
has, strong ownership

26. Polymorphism

Polymorphism means:

| **One interface, multiple implementations.**

Two major categories:

Compile-time polymorphism

- Function overloading
- Operator overloading
- Templates

Runtime polymorphism

- Virtual functions
- Method overriding

27. Function Overloading

Same function name, different parameters.

```
class Calculator {  
public:  
    int add(int a, int b) {  
        return a+b;  
    }  
    double add(double a, double b) {  
        return a+b;  
    }  
};
```

Compiler decides which function to call.

```
Calculator c;  
c.add(2,3);  
c.add(2.5,3.5);
```

This is:

| **Compile-time polymorphism.**

28. Operator Overloading

What is operator overloading?

C++ allows us to give an existing operator a meaning for our own class.

For example, normally:

```
int a =10;  
int b =20;  
int c=a+b;
```

+ already knows how to add integers.

But suppose we create:

```
class Point {  
public:  
int x;  
int y;  
};
```

What should this mean?

```
Point p1(1,2);  
Point p2(3,4);
```

```
Point p3 =p1+p2;
```

C++ doesn't automatically know how to add two `Point` objects.

We can define what `+` means.

```
class Point {
public:
    int x;
    int y;
    Point(int x,int y): x(x), y(y) {
    }
    Point operator+(const Point &other) {
        return Point(x+other.x,y+other.y);
    }
};
```

Now:

```
Point p1(1,2);
Point p2(3,4);
Point p3 =p1+p2;
cout<<p3.x<<" "<<p3.y;
```

Output:

```
4 6
```

The compiler effectively interprets:

```
p1+p2
```

as something similar to:


```
p1.operator+(p2)
```

So:

```
p1 + p2
  ↓
p1.operator+(p2)
```

Operators You Can Overload

Common operators:

```
+
-
*
/
%
==
!=
<
>
<=
>=
++
--
=
+=
-=
[]
()
->
<<
>>
```

You can overload many operators, but **not every operator**.

Operators You Cannot Overload

Important interview question.

You cannot overload:

```
::  
.  
.*  
?:  
sizeof  
typeid
```

For example:

```
a.b
```

cannot be given custom behavior through operator overloading.

Operator Overloading Syntax

General syntax:

```
return_type operatorOP(parameters){  
    // implementation  
}
```

Example:

```
Point operator+(const Point &other) {  
    ...  
}
```

Here:

```
Point  
↓  
return type  
  
operator+  
↓
```

operator being overloaded

const Point& other

↓

right-hand operand

Unary vs Binary Operators

This is a common interview question.

Binary operator

Works with two operands.

a+b

Examples:

+
-
*
/
==
<
>

For a member function:

a+b

becomes:

a.operator+(b)

So the left operand is implicitly the current object.

Unary operator

Works with one operand.

Examples:

```
-a++a  
a++--a
```

Example:

```
class Number {  
private:  
int value;  
public:  
Number(int value): value(value) {  
}  
Number operator-() {  
return Number(-value);  
}  
int getValue() {  
return value;  
}  
};
```

Usage:

```
Number n(10);  
Number result = -n;  
cout<<result.getValue();
```

Output:

```
-10
```

Prefix vs Postfix ++

Very common interview question.

Prefix

```
++x;
```

Postfix

```
x++;
```

C++ differentiates them using a dummy `int` parameter.

Prefix

```
Number &operator++() {  
    ++value;  
    return *this;  
}
```

Postfix

```
Number operator++(int) {  
    Number temp = *this;  
    value++;  
    return temp;  
}
```

Notice:

```
operator++()
```

vs:

```
operator++(int)
```

The `int` doesn't represent a meaningful value.

It tells the compiler:

| This is the postfix version.

== Operator Overloading

Suppose:

```
class Student {  
private:  
int id;  
public:  
Student(int id): id(id) {  
}  
bool operator==(const Student&other) const {  
return id==other.id;  
}  
};
```

Now:

```
Student s1(10);  
Student s2(10);  
if (s1==s2) {  
cout<<"Same student";  
}
```

The compiler effectively calls:

```
s1.operator==(s2);
```

[] Operator Overloading

You can make an object behave like an array.

```
class Array {  
private:  
int data[5];  
public:  
int&operator[](int index) {  
return data[index];  
}  
};
```

Now:

```
Array arr;  
arr[0]=10;  
arr[1]=20;  
cout<<arr[0];
```

This:

```
arr[0]
```

becomes approximately:

```
arr.operator[] (0)
```

Why Return `int&` ?

This is important.

If we wrote:

```
int operator[](int index)
```

then:

```
arr[0]=100;
```

wouldn't work because we're returning a temporary value.

With:

```
int& operator[](int index)
```

we return a reference to the actual element.

So:

```
arr[0]=100;
```

modifies the original data.

<< Operator Overloading

Very useful because you'll often see:

```
cout<<object;
```

Suppose:

```
class Point {  
public:  
    int x;  
    int y;  
    Point(int x,int y):x(x),y(y) {  
    }  
};
```


We can define:

```
ostream& operator<<(ostream& out, const Point&p) {  
    out<<" ("<<p.x<<" , "<<p.y<<" )";  
    return out;  
}
```

Now:

```
Point p(10,20);  
cout<<p;
```

Output:

```
(10, 20)
```

Why Is `operator<<` Usually Non-Member?

Notice:

```
ostream& operator<<(ostream& out, const Point &p)
```

The left operand is:

```
cout
```

which is an `ostream`.

So if this were a member of `Point`, it wouldn't naturally match:

```
cout<<p;
```

The operands are:

```
cout << p
  ↑      ↑
ostream Point
```

Therefore a non-member function is commonly used.

Operator Overloading ≠ Changing Precedence

Very important.

If you overload:

+

you don't change its precedence.

For example, `*` still has higher precedence than `+`.

Operator overloading changes **behavior**, not:

- precedence
- associativity
- number of operands

This is:

| **Compile-time polymorphism.**

29. Templates

Now the second major compile-time polymorphism concept.

A template allows you to write **generic code that works with different data types**.

Instead of:

```
int max(int a,int b);  
double max(double a,double b);  
long max(long a,long b);
```

we can write one function:

```
template<typename T>T maximum(T a,T b) {  
    return (a>b) ?a:b;  
}
```

Now:

```
cout<<maximum(10,20);  
cout<<maximum(2.5,1.5);  
cout<<maximum(100L,200L);
```

The compiler generates the appropriate version.

typename vs class

You will see:

```
template<typename T>
```

and:

```
template<class T>
```

Both are generally equivalent in this context.

```
template<typename T>
```

is often preferred because it makes it clear that `T` represents a type.

30. How Templates Work

Consider:

```
template<typename T>T add(T a,T b) {  
    return a+b;  
}
```

When you write:

```
add(10,20);
```

the compiler can instantiate something conceptually like:

```
int add(int a,int b) {  
    return a+b;  
}
```

When you write:

```
add(2.5,3.5);
```

it can instantiate:

```
double add(double a,double b) {  
    return a+b;  
}
```

So templates are resolved at **compile time**.

31. Function Templates

Basic:

```
template<typename T>T square(T value) {  
    return value*value;  
}
```

Usage:

```
cout<<square(5);  
cout<<square(2.5);
```

32. Multiple Template Parameters

```
template<typename T,typename U>void print(T a,U b) {  
    cout<<a<<" "<<b<<endl;  
}
```

Now:

```
print(10,2.5);
```

Here:

```
T = int  
U = double
```

33. Class Templates

Templates aren't only for functions.

You can create generic classes.

Example:

```
template<typename T> class Box {
private:
T value;
public:
Box(T value):value(value) {
}
T getValue() {
return value;
}
};
```

Now:

```
Box<int>int Box(10);Box<double>doubleBox(10.5);Box<string>stringBox("Hello");
```

Same class design, different types.

34. Template Function Overloading

You can combine templates with overloading.

```
template<typename T> void print(T value) {
cout<<value<<endl;
}
void print(string value){
cout<<"String: "<<value<<endl;
}
```

Then:

```
print(10);
```

and:

```
print("Hello");
```

can resolve differently.

35. Why is operator overloading polymorphism?

Because the same operator can behave differently depending on the types.

For example:

```
int a=10;  
int b=20;  
a+b;
```

means integer addition.

But:

```
Point p1(1,2);  
Point p2(3,4);  
p1+p2;
```

means our custom `Point` addition.

Same syntax:

```
+
```

Different behavior depending on the operand types.

That's **compile-time polymorphism**.

36. Templates vs Runtime Polymorphism

This distinction is **very important for interviews**.

Templates

```
template<typename T> void process(T obj);
```

The type is generally determined at **compile time**.

Virtual functions

```
class Animal {  
public:  
virtual void sound()=0;  
};
```

The implementation is selected at **runtime** through dynamic dispatch.

Think:

COMPILE-TIME

Function overloading
Operator overloading
Templates

RUNTIME

Virtual functions
Method overriding
Dynamic dispatch

Why Templates Are Called Static Polymorphism

Suppose:

```
template<typename T> void makeSound(T animal) {  
    animal.sound();  
}
```



```
}
```

You call:

```
Dog dog;  
makeSound(dog);
```

The compiler knows:

```
T = Dog
```

at compile time.

It can generate the appropriate code.

Therefore:

```
Templates  
  ↓  
Compile time  
  ↓  
Static polymorphism
```

38. Static vs Dynamic Polymorphism

Imagine a payment system.

Runtime polymorphism

```
class Payment {  
public:  
    virtual void pay() = 0;  
};  
class Stripe : public Payment {  
public:  
    void pay() override {  
        cout << "Stripe";  
    }  
};
```

```

    }
};
class PayPal : public Payment {
public:
void pay() override{
cout<<"PayPal";
    }
};

```

Then:

```

Payment* payment = new Stripe();
payment->pay();

```

The actual implementation is determined at runtime.

Compile-time polymorphism

With templates:

```

template <typename PaymentMethod> void processPayment(Payment
Method &payment) {
    payment.pay();
}

```

Then:

```

Stripe stripe;
PayPal paypal;
processPayment(stripe);
processPayment(paypal);

```

The compiler determines the types.

39. Function Overriding

Derived class provides its own implementation of a base-class method.

```
class Animal {
public:
virtual void sound() {
cout<<"Animal sound\n";
}
};
class Dog :
public Animal {
public:
void sound() override {
cout<<"Bark\n";
}
};
```

40. Runtime Polymorphism

The important part:

```
Animal* animal =newDog();
animal->sound();
```

Output:

Bark

Why?

Because `sound()` is virtual.

Without `virtual`:

```
class Animal {
public:
```

```
void sound() {  
    cout<<"Animal";  
}  
};
```

then:

```
Animal*animal =new Dog();  
animal->sound();
```

would call the base implementation.

41. Virtual Function

A virtual function allows C++ to perform **dynamic dispatch**.

```
class Animal {  
public:  
    virtual void sound() {  
        cout<<"Animal\n";  
    }  
};
```

Then:

```
Animal*a =newDog();  
a->sound();
```

The actual object determines which implementation executes.

42. Pure Virtual Function

```
class Shape {  
public:
```

```
virtual double area()=0;
};
```

This makes `Shape` an **abstract class**.

Derived class must implement it:

```
class Circle :public Shape {
private:
double radius;
public:
Circle(double r) : radius(r) {}
double area()override {
return 3.14159*radius*radius;
}
};
```

Two major types:

Compile-time polymorphism

- Function overloading
- Operator overloading
- Templates

Runtime polymorphism

- Virtual functions
- Function overriding

43. Abstract Class

A class containing at least one pure virtual function is abstract.

```
class Shape {
public:
    virtual double area() =0;
};
```

You cannot instantiate it:

```
Shape s; // ERROR
```

But:

```
Shape*s =newCircle();
```

is valid if `Circle` implements `area()`.

44. Interface in C++

C++ doesn't have a dedicated `interface` keyword like Java.

An interface can be modeled using a class containing pure virtual functions.

```
class Payment {
public:
    virtual void pay(double amount)=0;
    virtual void refund(double amount)=0;
    virtual ~Payment()=default;
};
```

Then:

```
class CreditCard: public Payment {
public:
    void pay(double amount) override {
        cout<<"Paid using card";
    }
};
```

```
void refund(double amount) override {  
    cout<<"Refunded";  
}  
};
```

45. Virtual Destructor

Very important interview question.

Consider:

```
class Base {  
public:  
    ~Base() {  
        cout<<"Base destructor";  
    }  
};  
class Derived: public Base {  
public:  
    ~Derived() {  
        cout<<"Derived destructor";  
    }  
};  
Base *ptr=new Derived();  
delete ptr;
```

If base destructor isn't virtual, deleting through a base pointer can result in the derived destructor not being invoked correctly, leading to resource leaks/undefined behavior depending on the design.

Correct:

```
class Base {  
public:
```

```
virtual ~Base()=default;
};
```

Then:

```
Base *ptr=new Derived();
delete ptr;
```

Destruction:

```
Derived destructor
Base destructor
```

Rule

If a class is intended to be used polymorphically, its destructor should generally be virtual.

46. Friend Function

A friend function is not a member of the class but can access its private/protected members.

```
class Box {
private:
int value;
public:
Box(int v):value(v) {
}
friend void print(Box b);
};
void print(Box b) {
cout<<b.value;
}
```


Important

Friendship is:

- Not inherited
- Not transitive
- Not automatically reciprocal

47. Friend Class

```
class B;  
class A {  
private:  
int x=10;  
friend class B;  
};  
class B {  
public:  
void show(A obj) {  
cout<<obj.x;  
}  
};
```

B can access private members of **A**.

48. Copy Constructor

Copy constructor creates an object from another object.

Syntax:

```
ClassName(const ClassName &other)
```

Example:

```
class Student {  
public:  
    string name;  
    Student(string n): name(n) {  
    }  
    Student(const Student &other) {  
        name=other.name;  
    }  
};
```

```
Student s1("Ali");  
Student s2=s1;
```

49. Copy Assignment Operator

Different from copy constructor.

```
Student s1("Ali");  
Student s2("Sara");  
s2=s1;
```

Here `s2` already exists.

This invokes:

```
operator=
```

Whereas:

```
Student s2=s1;
```

initializes a new object → copy constructor.

50. Rule of 3

If a class manually manages a resource and needs any one of these:

1. Destructor
2. Copy constructor
3. Copy assignment operator

it usually needs all three.

Example:

```
class Array {
private:
int* data;
public:
Array(int value) {
data=new int(value);
}
~Array(){
delete data;
}
Array(const Array &other) {
data =new int(*other.data);
}
Array &operator=(const Array &other) {
if (this!= &other) {
delete data;
data=new int(*other.data);
}
return *this;
}
};
```

51. Shallow Copy

Copies the pointer/address.

```
class A {  
public:  
    int* ptr;  
    A(int x) {  
        ptr=new int(x);  
    }  
};
```

Default copying can result in:

```
A -> ptr -> 10  
A -> ptr _____ ↑
```

Both objects point to the same memory.

This can cause:

```
double delete  
dangling pointers  
unexpected modifications
```

52. Deep Copy

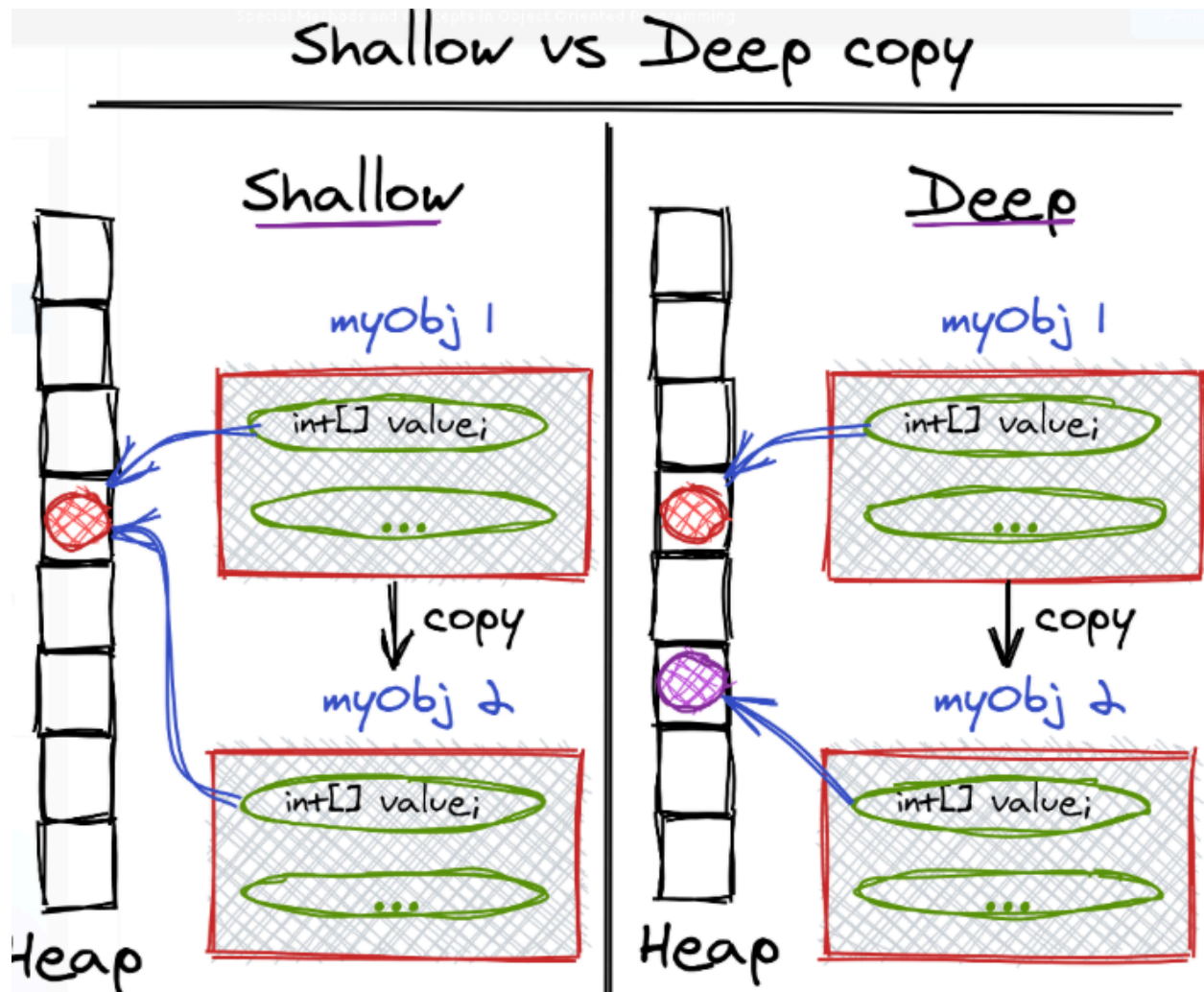
Creates independent memory.

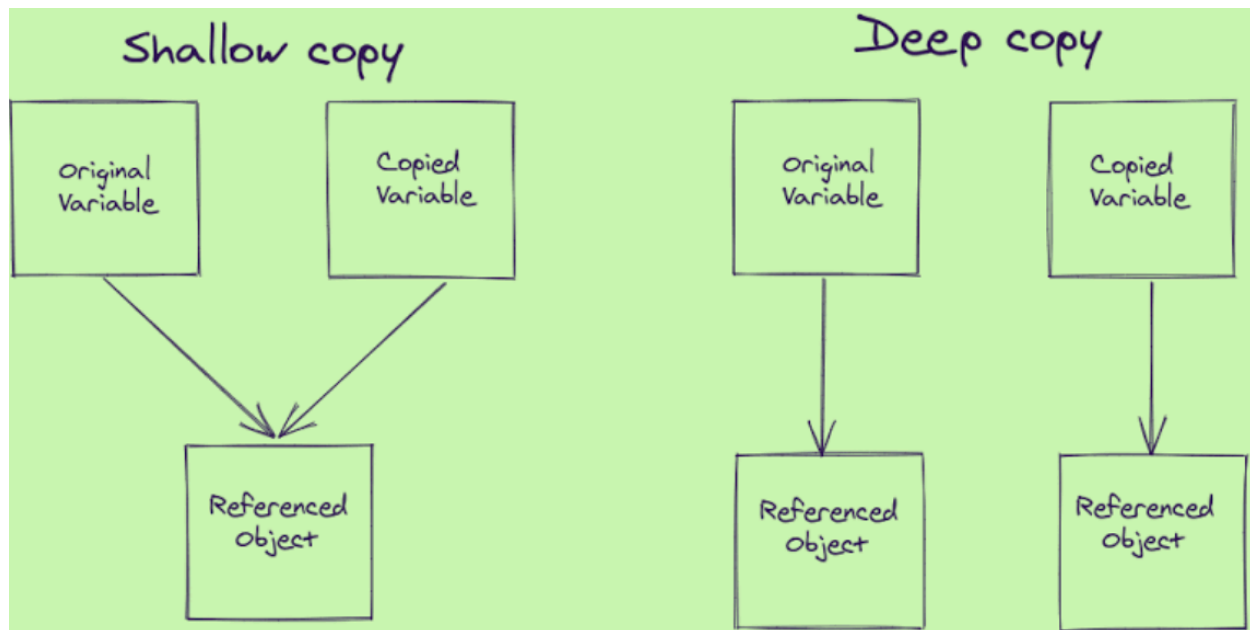
```
A(const A &other) {  
    ptr=new int(*other.ptr);  
}
```

Now:

Object A → 10
Object B → 10

but stored in different memory locations.





53. Rule of 5

Modern C++ adds:

1. Destructor
2. Copy constructor
3. Copy assignment
4. Move constructor
5. Move assignment

Example:

```
class Buffer {  
public:  
    Buffer(const Buffer &other);  
    // copy constructor  
    Buffer &operator=(const Buffer &other);  
    // copy assignment  
    Buffer(Buffer && other);  
    // move constructor  
    Buffer &operator=(Buffer &&other);
```

```
// move assignment  
~Buffer();  
// destructor  
};
```

54. Move Semantics

Move semantics allow resources to be **transferred instead of copied**.

Useful for temporary objects and expensive resources.

```
class Buffer {  
private:  
    int* data;  
public:  
    Buffer(int value) {  
        data=new int(value);  
    }  
    Buffer(Buffer &&other)noexcept {  
        data=other.data;  
        other.data=nullptr;  
    }  
    ~Buffer() {  
        delete data;  
    }  
};
```

Instead of copying the resource:

```
old → resource  
new → copy of resource
```

Move:

```
old → nullptr
```

```
new → original resource
```

55. `std::move`

`std::move` doesn't itself move anything.

It casts an expression so that move operations can be selected.

```
string a="Hello";  
string b=std::move(a);
```

The resources may be transferred from `a` to `b`.

After moving, `a` is valid but its value is generally **unspecified**.

56. Rule of Zero

Modern C++ encourages avoiding manual resource management when possible.

Use RAII types such as:

```
std::string  
std::vector  
std::unique_ptr  
std::shared_ptr
```

Then you often don't need to manually implement:

```
destructor  
copy constructor  
copy assignment  
move constructor  
move assignment
```


This is the **Rule of Zero**.

57. Upcasting

Converting derived object reference/pointer to base type.

```
Dog d;  
Animal *a=&d;
```

Safe and implicit.

58. Downcasting

Converting base pointer/reference to derived type.

```
Animal *a =new Dog();  
Dog *d=dynamic_cast<Dog*>(a);
```

`dynamic_cast` performs runtime-checked casting for polymorphic hierarchies.

Always check the result when using pointer form:

```
if (d!=nullptr) {  
    d->bark();  
}
```

59. `dynamic_cast`

Used for safe runtime downcasting.

```
Animal *animal=new Dog();  
Dog *dog=dynamic_cast<Dog*>(animal);
```

Requires the base class to be polymorphic, typically by having at least one virtual function.

60. `static_cast` vs `dynamic_cast`

`static_cast`

Compile-time conversion.

```
double x=10.5;
int y=static_cast<int>(x);
```

For class hierarchies, it doesn't perform the same runtime safety check as `dynamic_cast`.

`dynamic_cast`

Runtime-checked casting within polymorphic class hierarchies.

```
Dog *dog=dynamic_cast<Dog*>(animal);
```

61. Virtual vs Pure Virtual

Virtual

Has implementation:

```
virtual void draw() {
    cout<<"Drawing";
}
```

Derived class **may** override it.

Pure virtual

```
virtual void draw()==0;
```

Derived concrete class **must** override it.

62. Can a Pure Virtual Function Have an Implementation?

Yes.

```
class Animal {  
public:  
virtual void sound()==0;  
};  
void Animal::sound() {  
cout<<"Animal sound";  
}
```

A derived class can explicitly call it:

```
Animal::sound();
```

The class is still abstract because the function is pure virtual.

63. Static vs Virtual Functions

A static function cannot be virtual.

Why?

Virtual dispatch requires an object/runtime type, while static member functions belong to the class and have no `this` pointer.

Invalid:

```
static virtual void test();
```

64. Constructor Cannot Be Virtual

Why?

Virtual dispatch depends on the object already being constructed. During construction, the derived object isn't fully constructed yet.

Instead, C++ provides patterns such as:

- Factory pattern
- Virtual clone function

65. Object Slicing

Suppose:

```
class Animal {  
public:  
    virtual void sound() {  
        cout<<"Animal";  
    }  
};  
class Dog:public Animal {  
public:  
    void sound()override {  
        cout<<"Dog";  
    }  
};
```

Now:

```
Dog d;  
Animal a=d;
```

The derived portion gets sliced off.

```

Dog object
+-----+
| Animal portion |
| Dog portion    |
+-----+

      ↓ copy

Animal object
+-----+
| Animal portion |
+-----+

```

Avoid slicing using:

```
Animal&a =d;
```

or:

```
Animal*a = &d;
```

66. Early Binding vs Late Binding

Early binding

Resolved at compile time.

Examples:

```

Function overloading
Operator overloading

```

Late binding

Resolved at runtime.

Example:

```
Animal*a =newDog();a->sound();
```

when `sound()` is virtual.

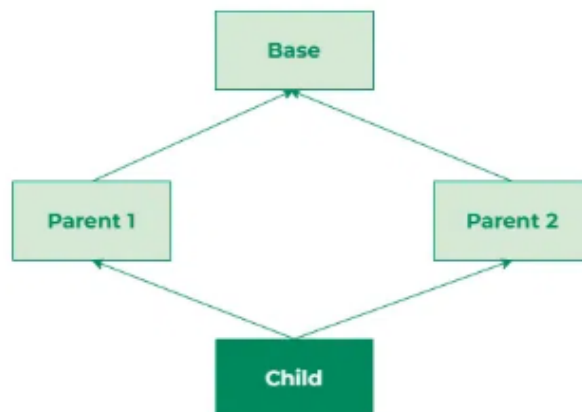
67. Virtual Function vs Function Overloading

Overloading	Overriding
Same class usually	Base + derived
Different parameters	Same signature
Compile-time	Runtime
No inheritance required	Requires inheritance
Static binding	Dynamic dispatch when virtual

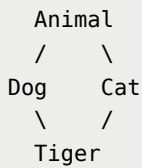
overloading can be done in same class as well as different class, overriding requires different derived class.

68. Diamond Problem

Diamond Problem in C++



Multiple inheritance:



If both `Dog` and `Cat` inherit from `Animal`, `Tiger` can get **two copies of Animal**.

```
class Animal {};  
class Dog: public Animal {};  
class Cat: public Animal {};  
class Tiger: public Dog,public Cat {};
```

Now:

```
Tiger t;
```

There are two `Animal` subobjects.

69. Virtual Inheritance

Solves the diamond duplication.

```
class Animal {};  
class Dog: virtual public Animal {};  
class Cat: virtual public Animal {};  
class Tiger: public Dog,public Cat {};
```

Now `Tiger` has one shared `Animal` base subobject.

70. Multiple Inheritance

C++ supports:

```
class C: publicA,publicB {};
```

Example:

```
class Printer {  
    public:  
    void print() {}  
};  
class Scanner {  
    public:  
    void scan() {}  
};  
class Machine: public Printer, public Scanner {};
```

71. Ambiguity in Multiple Inheritance

```
class A {  
    public:  
    void show() {}  
};  
class B {  
    public:  
    void show() {}  
};  
class C: public A, public B {};
```

This is ambiguous:

```
C obj;  
obj.show(); // ERROR
```

Resolve:

```
obj.A::show();
```


72. Const Member Function

A const member function promises not to modify the object's observable state through that member function.

```
class Student {
private:
int age;
public:
Student(int a) : age(a) {}
    // Const member function: read-only guarantee
    int getAge() const {
        return age;
    }
    // Non-const member function: may mutate state
    void setAge(int a) {
        age = a;
    }
};
```

You can call it on const objects:

```
const Student s;
s.getAge();
// s.setAge(21); // COMPILE ERROR: cannot call non-const method on const object
```

The Underlying Mechanism: The `this` Pointer

Inside every non-static member function of a class `Student`, an implicit `this` pointer is passed to refer to the calling object:

- In a **non-const member function** (`int getAge()`): `this` has the type `Student* const` (a constant pointer to a mutable `Student`).
- In a **const member function** (`int getAge() const`): `this` has the type `const Student* const` (a constant pointer to a `const Student`).

Because `this` points to a constant instance, the compiler forbids assigning values to members (e.g., `age = 20;` triggers a compile error) and prevents calling other non-const member functions through `this`.

Why Calling on `const` Objects Works

In C++, const-correctness enforces read-only access. When an object is instantiated as `const`, its state must remain immutable throughout its lifetime.

73. Const Object

```
const Student s;
```

A const object can only call member functions that are themselves `const` (subject to special cases such as mutable members).

74. Mutable Keyword

A `mutable` data member can be modified inside a `const` member function.

```
class Counter {
private:
mutable int calls=0;
public:
void show()const {
calls++;
cout<<calls;
}
};
```

Useful for caching/logging state that doesn't conceptually change the object's logical value.

75. Const Correctness

Very important in professional C++.

```
void print(const Student&s);
```

Means:

- Don't copy object
- Don't modify object

For expensive objects:

```
void process(const vector<int>&data);
```

is usually better than:

```
void process(vector<int> data);
```

when you don't need a copy.

76. Reference vs Pointer in OOP

Pointer:

```
Student*s;
```

Can be:

```
nullptr
```

Reference:

```
Student&s;
```

Must refer to an existing object once initialized.

Polymorphism commonly uses:

```
Animal*animal;
```

or:

```
Animal&animal;
```

77. RAII

Resource Acquisition Is Initialization

Resources are tied to object lifetime.

```
{  
    std::unique_ptr<int> p = std::make_unique<int>(10);  
}
```

When `p` goes out of scope, the managed resource is automatically released.

RAII is one of the most important C++ concepts for writing safe resource-management code.

Resources include:

- Memory
- Files
- Locks
- Sockets
- Database connections

78. Smart Pointers

Modern C++ provides:

```
unique_ptr  
shared_ptr  
weak_ptr
```

unique_ptr

One owner.

```
unique_ptr<int>p =make_unique<int>(10);
```

Cannot be copied:

```
unique_ptr<int>p2 =p;// ERROR
```

Can be moved:

```
unique_ptr<int>p2 = std::move(p);
```

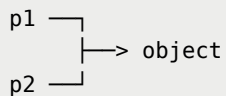
79. shared_ptr

Multiple owners.

```
shared_ptr<int>p1 =make_shared<int>(10);shared_ptr<int>p2 =p1;
```

Both share ownership.

Reference count conceptually becomes:



Object is destroyed when the last owning `shared_ptr` is destroyed/reset.

80. weak_ptr

Doesn't own the object.

Used to break `shared_ptr` cycles.

```
shared_ptr<Node>p1;weak_ptr<Node>weak =p1;
```

To access:

```
if (autop =weak.lock()) { // use p  
}
```

81. `shared_ptr` Cyclic Reference

Example:

```
A shared_ptr → B  
B shared_ptr → A
```

Both reference counts remain nonzero.

Therefore objects may never be destroyed.

Use:

```
weak_ptr
```

for the non-owning relationship.

82. Accessing Private Data Through Getter/Setter

```

class User {
private:
string password;
public:
void setPassword(const string &p) {
if (p.length()>=8)
password=p;
}
string getPassword()const {
return password;
}
};

```

But don't blindly expose sensitive/internal state through getters. Encapsulation is about exposing a useful interface, not automatically generating getters for everything

83. Coupling and Cohesion

Coupling

How dependent one class is on another.

Low coupling is generally desirable.

Cohesion

How closely related the responsibilities inside a class are.

High cohesion is desirable.

Good OOP design generally aims for:

Low Coupling
High Cohesion

Dependency Injection

Instead of creating dependencies internally:

```
class Service {  
    private:  
    Database db;  
    public:  
    Service() { // tightly coupled  
        }  
};
```

Inject dependency:

```
class Service {  
    private:  
    Database& db;  
    public:  
    Service(Database& database): db(database) {}  
};
```

This improves:

- Testing
- Flexibility
- Maintainability
- Loose coupling

Important Interview Trap

What is the output?

```
class Base {  
    public:  
    void show() {
```



```
cout<<"Base";
    }
};
class Derived: public Base {
public:
void show() {
cout<<"Derived";
    }
};
Base*b=new Derived();
b->show();
```

Answer:

Base

Because `show()` is not virtual.

Now:

```
class Base {
public:
virtual void show() {
cout<<"Base";
    }
};
```

Output:

Derived

Another Common Interview Question

```

class Base {
public:
    virtual ~Base() {
        cout<<"Base";
    }
};
class Derived: public Base {
public:
    ~Derived() {
        cout<<"Derived";
    }
};
int main() {
    Base*b=new Derived();
    delete b;
}

```

Output:

```

DerivedBase

```

Because destructors execute derived → base.

Constructor Calling Virtual Function

This is a common trap.

```

class Base {
public:
    Base() {
        show();
    }
    virtual void show() {
        cout<<"Base";
    }
};

```

```

    }
};
class Derived: public Base {
public:
void show() override {
cout<<"Derived";
    }
};

```

When constructing `Derived`, the base constructor runs first.

During the base constructor, virtual dispatch does **not** dispatch to the not-yet-constructed derived part.

So it calls:

```
Base
```

Similarly, calling virtual functions from destructors does not dispatch to already-destroyed derived parts.

Best practice: avoid calling virtual functions from constructors/destructors.

Constructor Delegation

One constructor can call another constructor.

```

class Student {
private:
string name;
int age;
public:
Student():Student("Unknown",0) {
}
Student(string n,int a): name(n), age(a) {

```

```
}  
};
```

final

Prevents overriding:

```
class Base {  
public:  
virtual void show() final {}  
};
```

Derived class cannot override it.

Can also prevent inheritance:

```
class Base final {};
```

Now:

```
class Derived: public Base {};
```

is invalid.

Nested Classes

A class can be defined inside another class.

```
class Outer {  
public:  
class Inner {  
public:  
void show() {}  
};  
};
```

Usage:

```
Outer::Inner obj;
```

Anonymous Objects

You can create temporary objects:

```
Student("Ali", 20);
```

Often useful when passing/returning temporary values.

Object Lifetime

Typical object lifecycle:

```
Memory acquired
  ↓
Constructor
  ↓
Object used
  ↓
Destructor
  ↓
Memory/resource released
```

For local objects:

```
void test() {
    Student s;
}
```

Destructor automatically runs when `s` leaves scope.

Stack vs Heap Objects

Stack:

```
Student s;
```

Automatic lifetime tied to scope.

Heap:

```
Student*s =new Student();  
delete s;
```

Manual lifetime management if using raw `new/delete` .

```
Raw pointer  
  ↓  
Ownership?  
  ↓  
unique_ptr — one owner  
shared_ptr — shared ownership  
weak_ptr — non-owning reference  
  ↓  
RAII  
  ↓  
Rule of Zero
```

Most Important Interview Questions

You should be able to answer these without hesitation:

Basics

1. What is OOP?
2. What are the four pillars?
3. Class vs object?
4. Encapsulation vs abstraction?

5. What are access modifiers?
6. What is `this` ?
7. What is a constructor?
8. What is a destructor?
9. Can constructors be overloaded?
10. Can destructors be overloaded?

Inheritance

1. What is inheritance?
2. Types of inheritance?
3. What is multiple inheritance?
4. What is the diamond problem?
5. What is virtual inheritance?
6. Public vs private vs protected inheritance?
7. IS-A vs HAS-A?

Polymorphism

1. What is polymorphism?
2. Compile-time vs runtime polymorphism?
3. Overloading vs overriding?
4. What is a virtual function?
5. What is a pure virtual function?
6. What is an abstract class?
7. Why use `override` ?
8. Why can't static functions be virtual?
9. Why can't constructors be virtual?
10. Why should polymorphic bases have virtual destructors?

Memory / Objects

1. Copy constructor?
2. Copy assignment?
3. Shallow vs deep copy?
4. Rule of 3?
5. Rule of 5?
6. Rule of 0?
7. Move constructor?
8. Move assignment?
9. `std::move` ?
10. RAI?
11. Smart pointers?
12. `unique_ptr` vs `shared_ptr` vs `weak_ptr` ?
13. What is a cyclic reference?

Advanced

1. What is object slicing?
2. What is vtable?
3. What is vptr?
4. What is dynamic dispatch?
5. What is `dynamic_cast` ?
6. `static_cast` vs `dynamic_cast` ?
7. What is composition?
8. Aggregation vs composition?
9. What is association?
10. Why favor composition over inheritance?

Design

1. Explain SOLID.
2. Explain dependency injection.
3. Design a payment system.
4. Design a notification system.
5. Design a parking lot.
6. Design a library management system.
7. Design a logging system.